Reporte Técnico de Actividades Práctico-Experimentales
Nro. 004

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Miguel Alejandro Armas Ordoñez Royel Ivan Jima Pardo Ana Cristina Panamito Flores Jostin Xavier Vasquez Calderón
Asignatura	Simulación
Ciclo	5 A
Unidad	1
Resultado de aprendizaje de la unidad	R1. Aplica análisis estadísticos de los datos de la Simulación para mejorar la calidad de los resultados, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	004
Título de la Práctica	Análisis comparativo de herramientas de simulación mediante un modelo común
Nombre del Docente	Ing. José O. Guamán Q
Fecha	22 de Octubre de 2026 23 de Octubre 2026
Horario	Jueves 10h30 a 11h30 Viernes 07h30 a 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica:

Comparar diferentes lenguajes y plataformas para la implementación de modelos de simulación.

- Implementar un modelo sencillo en diferentes herramientas.
- Comparar sintaxis, facilidad de implementación y visualización.
- Identificar el ámbito de aplicación de cada herramienta

3. Materiales y reactivos:

- Computador.
- Python.

- NetLogo.
- Java/JDK.

4. Equipos y herramientas

- SimPy
- NumPy
- Matplotlib.
- NetLogo.
- JDK.
- Sistema operativo Linux o macOS

5. Procedimiento / Metodología

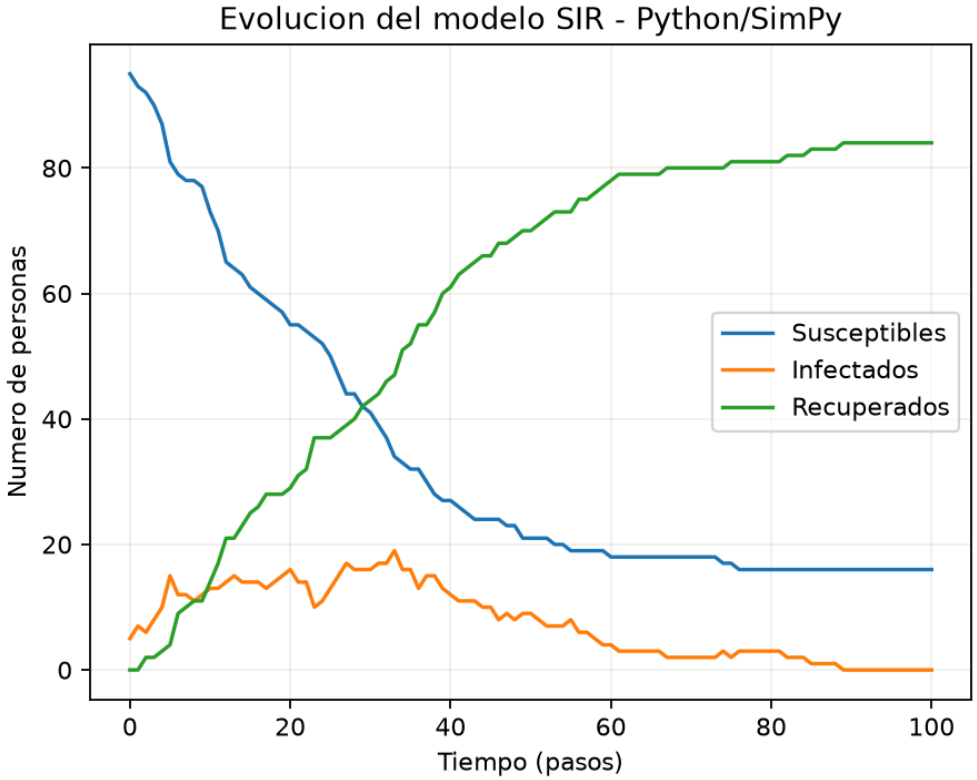
Se estableció un modelo matemático epidemiológico base con una población total de $N = 100$ individuos, divididos inicialmente en $S = 95$ susceptibles, $I = 5$ infectados y $R = 0$ recuperados. Los parámetros de control definidos fueron una tasa de infección $\beta = 0.30$ y una tasa de recuperación $\gamma = 0.10$, con un número reproductivo básico $R_0 = 3.00$ y un tiempo total de simulación de 100 pasos.

Se desarrolló el modelo en Python aplicando una arquitectura por capas. Se utilizó la biblioteca SimPy para gestionar los eventos discretos en intervalos de tiempo y números pseudoaleatorios para el cálculo estocástico de las transiciones de estado de cada persona.

Se automatizó la generación de un archivo de resultados en formato plano que almacena las columnas de tiempo, susceptibles, infectados y recuperados en cada paso de la simulación

Se creó el modelo equivalente en NetLogo configurando la interfaz gráfica, los botones de ejecución, las series de trazos en las gráficas y el procedimiento para importar y representar gráficamente los datos exportados previamente desde Python.

6. Resultados Esperados



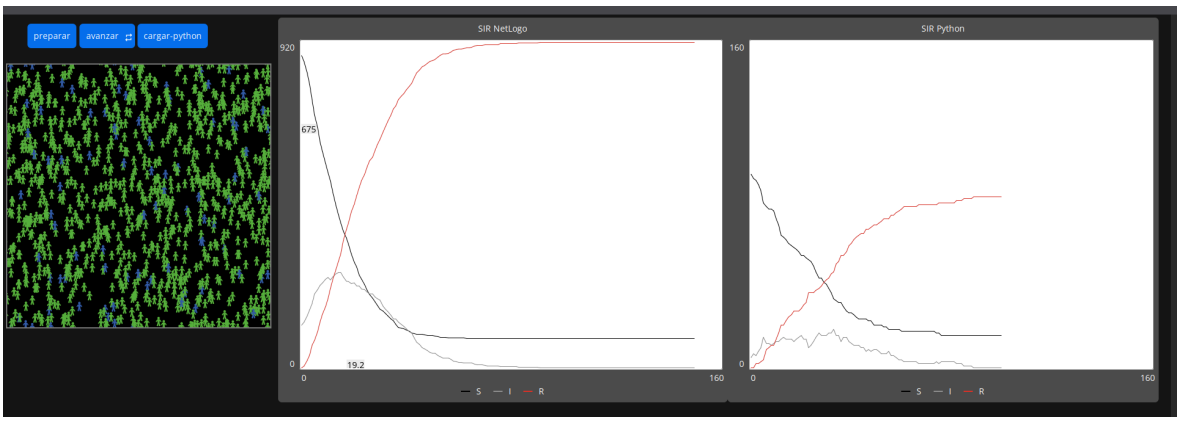

```
================================================
  SIMULACION DEL MODELO EPIDEMIOLOGICO SIR
================================================
Desea usar los valores por defecto de la guia? (s/n) [s]: s

=== DATOS INICIALES ===
Poblacion total N = 100
Valores iniciales: S = 95, I = 5, R = 0
Beta = 0.3, Gamma = 0.1, Semilla = 42
Cambios esperados en t = 0: dS = -1.425, dI = 0.925, dR = 0.500
Numero reproductivo basico R0 = 3.00

Ejecutando la simulacion...
Verificacion correcta: en todas las filas S + I + R = 100

=== RESULTADOS ===
Filas generadas: 101
Pico de infectados: 19 personas en el tiempo 33
Estado final -> tiempo=100 S=16 I=0 R=84

=== ARCHIVOS GENERADOS ===
CSV para NetLogo : /home/xdylok/Proyectos/Python/Simulacion/Practica#4Grupal/netlogo/resultados_python.csv
CSV de Euler     : /home/xdylok/Proyectos/Python/Simulacion/Practica#4Grupal/netlogo/resultados_euler_python.csv
Grafica PNG      : /home/xdylok/Proyectos/Python/Simulacion/Practica#4Grupal/resultados/grafica_python.png
```



Interpretación:

Susceptibles (azul): Comienzan en su punto máximo (95 personas) y descienden de manera constante a medida que el virus se propaga y la gente se contagia, estabilizando finalmente en 16 personas que lograron no contraer la enfermedad al final de la simulación.

Infectados (naranjada): Muestran fluctuaciones típicas de un modelo estocástico/discreto. La curva asciende hasta alcanzar un pico máximo de 19 personas infectadas en el tiempo 33, momento crítico donde la epidemia está más activa, para luego descender paulatinamente hasta extinguirse ($I = 0$ al final).

Recuperados (verde): Crecen de forma inversa a los susceptibles. Comienzan en cero y aumentan conforme los infectados sanan, convirtiéndose en el grupo mayoritario y estabilizándose en 84 personas al concluir el periodo.

Repositorio de la Práctica:

https://github.com/jxvasquez/Simulacion/

7. Preguntas de Control

1. **¿Qué representa el modelo SIR?**

   El modelo SIR representa un marco de simulación epidemiológico y de transmisión que describe cómo una enfermedad contagiosa o un rasgo cultural se propaga dentro de una población a través de tres estados fundamentales de los individuos [1].

2. **¿Qué representan los estados S, I y R?**

   S (Susceptible): Representa a los individuos sanos que están expuestos o son potencialmente receptores de la enfermedad o rasgo cultural [2].

   I (Infectado o Portador): Representa a los individuos enfermos o activos que transmiten la infección o propagan la innovación/rasgo a sus vecinos [2].

   R (Recuperado o Removido): Representa a los individuos que se han recuperado y adquirido inmunidad, que han fallecido o que han rechazado el rasgo, dejando de transmitir la infección

3. **¿Qué función cumplen los parámetros β y γ?**

   β **(Tasa de transmisión/infección):** Define la probabilidad o velocidad con la que un individuo infectado transmite la enfermedad o rasgo a un individuo susceptible tras un contacto [3].

γ **(Tasa de recuperación/remoción):** Define la tasa o probabilidad por unidad de tiempo a la que un individuo infectado se recupera, adquiere inmunidad o deja de estar en estado activo de contagio [3].

4. **¿Por qué es importante utilizar los mismos parámetros en las tres herramientas?**

   Utilizar los mismos parámetros garantiza la consistencia y comparabilidad experimental entre las diferentes plataformas de simulación. Esto nos asegura que cualquier variación observada en los resultados se deba a diferencias algorítmicas o a la arquitectura de la herramienta y no a errores en las entradas o condiciones iniciales del modelo [4]

5. **¿Por qué Python, Java y NetLogo pueden producir resultados diferentes utilizando el mismo modelo?**

   Pueden producir resultados diferentes debido a factores de variabilidad estocástica y técnica como la implementación de distintos generadores de números pseudoaleatorios, los mecanismos de programación y orden de ejecución de los agentes, y las divergencias en la precisión y cálculo aritmético de coma flotante entre plataformas [4].

6. **¿Qué ventajas ofrece NetLogo para representar sistemas basados en agentes?**

   NetLogo ofrece un entorno gráfico e interactivo diseñado específicamente para el modelado basado en agentes (ABM), con una sintaxis accesible y clara, abstracciones nativas para los agentes, el entorno, las redes y el observador, además de herramientas integradas para captura de datos, extensiones GIS/redes y visualización en tiempo real [5].

7. **¿Qué ventajas ofrece SimPy para desarrollar simulaciones basadas en eventos?**

   SimPy permite una gestión eficiente del tiempo al saltar directamente al instante del siguiente evento programado en lugar de evaluar condiciones en cada incremento de tiempo (tick). Lo que reduce la carga computacional, optimiza el modelado de colas con períodos de inactividad y facilita la integración con bibliotecas de análisis de datos en Python [4].

8. **¿Qué ventajas ofrece Java para implementar modelos de simulación?**

   Java ofrece un alto rendimiento en la ejecución, portabilidad multiplataforma gracias a la Máquina Virtual de Java, un soporte para Programación Orientada a Objetos, precisión matemática estricta y capacidades avanzadas para automatizar o controlar simulaciones mediante APIs de integración [4].

9. **¿Qué función cumple el archivo CSV en la integración Python/Java → NetLogo?**

   El archivo CSV actúa como un formato estándar de intercambio de datos en texto plano. Permite exportar datos, parámetros o configuraciones generadas en un entorno e importarlas en NetLogo mediante primitivas como csv:from-file, así como exportar los resultados simulados para su posterior análisis estadístico [5].

10. **¿Qué ventajas presenta separar la generación de datos de la visualización?**

    Separar la generación de datos de la visualización optimiza el rendimiento al ejecutar simulaciones en modo sin interfaz o en entornos de alto rendimiento, evitando la sobrecarga gráfica del renderizado en tiempo real. También, permite desacoplar los experimentos y emplear herramientas especializadas de ciencia de datos para realizar un análisis más profundo y personalizado [6].

8. Conclusiones

- Implementar el modelo SIR en distintas plataformas demostró que al utilizar las mismas ecuaciones y parámetros base, siempre habrá pequeñas variaciones en los resultados numéricos. Esto ocurre por el comportamiento estocástico y la forma en que cada lenguaje maneja sus propios generadores de números aleatorios.
- Aplicar una arquitectura por capas en Python fue importante para organizar el proyecto lo que permitió separar la lógica matemática de la interfaz y facilitó el mantenimiento del código como la exportación limpia de los datos en formato CSV para integrarlos sin problemas en NetLogo.

9. Bibliografía

[1] J. R. Baxter, *Introduction to Probability*. Minneapolis, MN, USA: University of Minnesota, 2024
[2] I. Romanowska, C. D. Wren, y S. A. Crabtree, *Agent-Based Modeling for Archaeology: Simulating the Complexity of Societies*. Santa Fe, NM, USA: The Santa Fe Institute Press, 2021
[3] D. P. Kroese y Z. I. Botev, *An Advanced Course in Probability and Stochastic Processes*, 2025[8]
[4] A. Chatterjee, Q. Cao, A. Sajadi, y B. Ravandi, "Deterministic random walk model in NetLogo and the identification of asymmetric saturation time in random graph," *Applied Network Science*, vol. 8, no. 1, art. 33, pp. 1–11, 2023, doi: 10.1007/s41109-023-00559-2
[5] U. Wilensky, *The NetLogo 7.0.4 User Manual*. Evanston, IL, USA: Center for Connected Learning and Computer-Based Modeling, Northwestern University, 2025[16]
[6] S. Moka, C. Hirsch, V. Schmidt, y D. P. Kroese, "Efficient Rare-Event Simulation for Random Geometric Graphs via Importance Sampling," *Research Article*, 2025

10. Declaración de IA

Modelo usado: Gemini 3.5 Pro

Durante el desarrollo de la presente Actividad Práctica Experimental (APE), se empleó una herramienta de Inteligencia Artificial (Gemini) como recurso de apoyo académico y técnico. Su utilización se estructuró en tres fases metodológicas clave:Verificación y depuración: Análisis y resolución de errores de ejecución e inconsistencias sintácticas surgidas durante la lectura de archivos de datos y la configuración de entornos.Refactorización de código: Asistencia en la optimización y estructuración del código base de Python bajo una arquitectura limpia por capas, asegurando la compatibilidad en la exportación de resultados en formato CSV para su procesamiento externo.Interpretación y sincronización gráfica: Soporte analítico para el ajuste de escalas, rangos de ejes (X/Y) y calibración de parámetros epidemiológicos (tasa de infección β, tasa de recuperación γ y población total N) con el fin de lograr una correcta convergencia visual y comparativa entre las curvas generadas en Python y el entorno de NetLogo.Todo el código resultante, la validación de resultados experimentales y la integración final de la plataforma fueron revisados, probados y adaptados de manera autónoma por el estudiante.

Prompts:

“Tengo un problema al ejecutar NetLogo y cargar el archivo CSV. Me sale un error de ejecución que dice 'There is no pen named S in the current plot' y posteriormente un error con 'FOREACH expected input to be a list but got the number 0'. ¿A qué se deben estos errores y cómo debo configurar exactamente los botones y los trazos de las gráficas para solucionarlo?”

“Actúa como profesor de programación y desarrollador experto en Python. Necesito que refactorices el código del modelo SIR adjunto en el proyecto, aplicando estrictamente una arquitectura por capas (Dominio, Aplicación, Infraestructura y Presentación). El código debe ser claro, estructurado para un nivel de principiante, comentado en español sin tildes, y asegurar la generación automática de un archivo CSV limpio y compatible para ser importado directamente en NetLogo."